



Chương 6

Xác thực và phân quyền

6.1. Xác thực bằng JWT

6.2. Phân quyền

6.3. OAuth2

6.4. Thực hành tạo Web app về phân quyền và đăng nhập

- **JWT** (JSON Web Token) là một tiêu chuẩn mã hóa gọn nhẹ dựa trên JSON, được sử dụng để truyền thông tin giữa các bên một cách an toàn.
- JWT thường được dùng để xác thực và ủy quyền (authentication and authorization) trong các ứng dụng web hoặc API.

Lợi ích của JWT:

- Gọn nhẹ, dễ dàng truyền qua HTTP Headers.
- Độc lập với công nghệ (cross-platform).
- Có thể mã hóa và/hoặc ký để đảm bảo tính an toàn và tin cậy.

JWT bao gồm 3 phần, được phân tách bởi dấu chấm (.):

- Header
- Payload
- Signature

- **Quy trình:**

- ✓ Client gửi yêu cầu đăng nhập với thông tin đăng nhập (username/password).
- ✓ Server xác thực thông tin. Nếu hợp lệ, server tạo JWT và trả về client.
- ✓ Client gửi JWT trong các yêu cầu tiếp theo (thường trong HTTP Header Authorization: Bearer <JWT>).
- ✓ Server kiểm tra JWT để quyết định cấp quyền truy cập tài nguyên.

Trong ASP.NET Core, JWT thường được sử dụng để xác thực API.

1. Cài đặt thư viện:

```
dotnet add package  
Microsoft.AspNetCore.Authentication.JwtBearer
```

2. Cấu hình JWT Middleware:

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
        .AddJwtBearer(options =>
        {
            options.TokenValidationParameters = new
TokenValidationParameters
            {
                ValidateIssuer = true,
                ValidateAudience = true,
                ValidateLifetime = true,
                ValidateIssuerSigningKey = true,
                ValidIssuer = "myApp",
                ValidAudience = "myAudience",
            }
        })
}
```


2. Cấu hình JWT Middleware (tiếp):

```
IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes("mySecretKey"))
    };
});
services.AddControllers();
}
public void Configure(IApplicationBuilder app, IWebHostEnvironment env)
{
    app.UseRouting();
    app.UseAuthentication();
    app.UseAuthorization();
    app.UseEndpoints(endpoints => endpoints.MapControllers());
}
```

3. Tạo JWT trong Controller:

```
[HttpPost("login")]
public IActionResult Login([FromBody] UserLogin model)
{
    if (model.Username == "admin" && model.Password == "password")
    {
        var claims = new[]
        {
            new Claim(ClaimTypes.Name, model.Username),
            new Claim(JwtRegisteredClaimNames.Sub, model.Username)
        };

        var key = new
        SymmetricSecurityKey(Encoding.UTF8.GetBytes("mySecretKey"));
        var creds = new SigningCredentials(key,
        SecurityAlgorithms.HmacSha256);
        var token = new JwtSecurityToken(
```

3. Tạo JWT trong Controller (tiếp):

```
issuer: "myApp",  
    audience: "myAudience",  
    claims: claims,  
    expires: DateTime.Now.AddHours(1),  
    signingCredentials: creds);  
  
        return Ok(new { token = new  
JwtSecurityTokenHandler().WriteToken(token) });  
    }  
    return Unauthorized();  
}
```

- **Xác thực (Authentication):** Kiểm tra danh tính người dùng (username/password hoặc token hợp lệ).
- **Phân quyền (Authorization):** Xác định quyền của người dùng để truy cập tài nguyên, thường dựa trên:
 - **Roles** (Vai trò): Định nghĩa cấp bậc (Admin, User...).
 - **Claims** (Thuộc tính): Chi tiết hóa các quyền (can_edit, can_delete...).
- **JWT** hỗ trợ tích hợp cả roles và claims để thực hiện phân quyền.

- **Server tạo JWT:**

- Bao gồm roles và claims trong **payload** của JWT.
- Ký JWT bằng **private key** hoặc **shared secret**.

- **Client gửi JWT:**

- JWT được đính kèm trong mỗi request, thường trong HTTP Header:
Authorization: Bearer <JWT>.

- **Server xác minh JWT:**

- Kiểm tra tính hợp lệ của token và trích xuất roles/claims.
- Áp dụng logic phân quyền để quyết định cho phép truy cập hay từ chối.

Sử dụng **Spring Boot** để tích hợp phân quyền vào REST API.

4.1. Cài đặt các dependency cần thiết

- Thêm các thư viện Spring Security và JWT:

2. Cấu hình Security và JWT

- Cấu hình Spring Security để phân quyền bằng JWT.

3. Custom Authorization Filter

Xây dựng một bộ lọc để kiểm tra JWT và phân quyền.
`javaCopy code`

4. Controller Phân quyền

- Sử dụng annotation để kiểm tra quyền.

```
@RestController
@RequestMapping("/api")
public class ApiController {
    @GetMapping("/admin")
    @PreAuthorize("hasRole('ADMIN')")
    public String adminAccess() {
        return "Admin Access Granted!";
    }
    @GetMapping("/user")
    @PreAuthorize("hasAnyRole('USER', 'ADMIN')")
    public String userAccess() {
        return "User Access Granted!";
    }
}
```

- OAuth2 là một giao thức ủy quyền phổ biến được sử dụng để cho phép các ứng dụng truy cập vào tài nguyên của người dùng mà không cần biết thông tin đăng nhập của họ
- Giao thức có mức độ triển khai rộng rãi trong việc xác thực và ủy quyền từ các ứng dụng web, ứng dụng di động, ứng dụng máy tính và các dịch vụ web khác

- OAuth2 là một giao thức xác thực và ủy quyền mở, cho phép ứng dụng bên thứ ba truy cập tài nguyên trên máy chủ API thay mặt người dùng mà không cần chia sẻ thông tin đăng nhập.
- Thường được sử dụng bởi các dịch vụ lớn như Google, Facebook, GitHub để cấp quyền truy cập vào API.

- OAuth2 hoạt động bằng cách cung cấp một phương pháp cho ứng dụng yêu cầu và nhận token ủy quyền từ một dịch vụ xác thực. Sau đó, nó sử dụng token này để truy cập vào tài nguyên được ủy quyền. Giao thức này tập trung vào việc truy cập vào tài nguyên người dùng từ các ứng dụng bên thứ ba mà không cần chia sẻ thông tin đăng nhập.

- Về vai trò cơ bản, OAuth2 hỗ trợ kiểm soát quyền truy cập, bảo mật thông tin của người dùng thông qua việc cung cấp phương pháp an toàn để xác thực và ủy quyền vào các ứng dụng, dịch vụ

- Xác thực và ủy quyền an toàn: OAuth2 cung cấp cơ sở xác thực người dùng an toàn và cấp quyền truy cập cho ứng dụng bên thứ ba mà không cần chia sẻ thông tin đăng nhập nhạy cảm như mật khẩu.
- Ứng dụng đa nền tảng: Hệ thống được sử dụng rộng rãi trong ứng dụng web, di động, máy tính và các dịch vụ web khác. Điều này làm cho nó trở thành một giao thức linh hoạt và phù hợp với các kiểu ứng dụng khác nhau.

- Quản lý quyền truy cập: Giao thức này cho phép người dùng kiểm soát cách mà các ứng dụng có thể truy cập vào tài nguyên của họ và họ có thể rút quyền truy cập bất kỳ lúc nào.
- Tích hợp dịch vụ bên thứ ba: OAuth2 hỗ trợ các dịch vụ khác nhau tương tác với nhau một cách an toàn và bảo mật. Công nghệ cũng góp phần nâng cao trải nghiệm cho người dùng thông qua tính tích hợp và chia sẻ thông tin.
- Bảo mật thông tin người dùng: Hệ thống có tác dụng bảo vệ thông tin cá nhân của người dùng bằng cách loại bỏ việc chia sẻ mật khẩu giữa các ứng dụng khác nhau.

Nguyên tắc hoạt động



- Xác thực ban đầu: Người dùng yêu cầu truy cập vào tài nguyên từ ứng dụng bên thứ ba. Ứng dụng chuyển hướng người dùng đến dịch vụ xác thực để đăng nhập và xác thực danh tính của họ.
- Cấp phép: Sau khi xác thực thành công, người dùng được hỏi xác đồng ý cho ứng dụng bên thứ ba truy cập vào tài nguyên của họ. Nếu người dùng đồng ý, một mã thông báo phép sẽ được tạo ra.

- Nhận mã thông báo phép: Ứng dụng bên thứ ba sẽ gửi mã thông báo phép đến dịch vụ xác thực và đổi lấy một mã thông báo truy cập và một mã thông báo làm mới.
- Truy cập vào tài nguyên: Mã thông báo truy cập sẽ được sử dụng để truy cập vào tài nguyên từ phía của ứng dụng bên thứ ba mà không cần phải biết thông tin đăng nhập của người dùng.

- Người dùng truy cập vào Ứng dụng và yêu cầu quyền truy cập vào Resource Server (ví dụ: Gmail, Facebook, Twitter, Github).
- Ứng dụng yêu cầu quyền truy cập thông qua người dùng, sau đó nhận được một chuỗi mã thông báo (authorization code) từ phía người dùng.
- Ứng dụng gửi chuỗi mã thông báo (authorization code) và thông tin định danh của mình (ID) kèm theo quyền của người dùng tới Authorization Server.

- Authorization Server xác thực chuỗi mã thông báo và thông tin định danh, sau đó cấp access token cho Ứng dụng (client) nếu mọi thông tin xác thực hợp lệ.
- Đã có access token, Ứng dụng (client) có thể sử dụng token này để truy cập vào Resource Server và lấy thông tin từ tài nguyên (resource).
- Nếu access token hợp lệ, Resource Server sẽ trả về dữ liệu của tài nguyên được yêu cầu cho Ứng dụng (client)

Ưu điểm

- An toàn: OAuth2 loại bỏ việc chia sẻ thông tin đăng nhập và mật khẩu giữa ứng dụng, người dùng và dịch vụ bên thứ ba, tạo ra một cách an toàn và bảo mật hơn để ủy quyền và truy cập dữ liệu.
- Linh hoạt: Hệ thống hỗ trợ nhiều loại ứng dụng khác nhau như ứng dụng web, di động và dịch vụ, làm cho nó trở thành giao thức ủy quyền phổ biến và linh hoạt.

Ưu điểm

- Quản lý quyền truy cập: Người dùng có khả năng kiểm soát cách mà các ứng dụng có thể truy cập vào tài nguyên của họ và họ có thể rút quyền truy cập bất kỳ lúc nào, tăng tính bảo mật và quyền riêng tư.
- Tích hợp dịch vụ bên thứ ba: OAuth2 cho phép tích hợp giữa các dịch vụ bên thứ ba một cách an toàn và bảo mật, giúp cung cấp trải nghiệm người dùng tốt hơn thông qua các tính năng cơ bản.

Nhược điểm

- Phức tạp cho việc triển khai: Quá trình vận dụng OAuth2 đòi hỏi người dùng cần hiểu biết kỹ thuật sâu về việc triển khai và quản lý giao thức. Điều này có thể làm tăng khả năng phát sinh lỗi đối với người dùng mới và cần có đội ngũ kỹ thuật dày dặn kinh nghiệm.
- Quản lý phiên: OAuth2 không đi kèm với một cơ chế quản lý phiên tích hợp sẵn. Đây là nguyên nhân tạo ra nhiều thách thức trong việc quản lý session và xác thực.

Roles trong OAuth2

- 1.Resource Owner** (Chủ sở hữu tài nguyên).
- 2.Client** (Ứng dụng khách).
- 3.Authorization Server** (Máy chủ ủy quyền).
- 4.Resource Server** (Máy chủ tài nguyên).

- **Tương tác:**

1. Resource Owner cấp phép cho Client thông qua Authorization Server.
2. Client lấy Access Token từ Authorization Server.
3. Client sử dụng Access Token để yêu cầu tài nguyên từ Resource Server.



CMC UNIVERSITY

Aspire to Inspire the Digital World

THANK YOU